Relatório de Auditoria de Segurança

Projeto: GENERICO

Versão:	1.0
Data:	2026-10-17
Autor:	Kiro AppSec
Tipo de análise:	Estática • White-box • Manual
Escopo:	Backend (Java/Spring Boot) · Frontend (Angular) · Banco (SQL Server) · CI/CD · Docke

■ CLASSIFICAÇÃO: CONFIDENCIAL

Este documento contém informações sensíveis sobre a postura de segurança do sistema. Acesso restrito à equipe de desenvolvimento e gestão autorizada.

Sumário

1. Resumo Executivo
2. Escopo, Metodologia e Limitações
3. Pontos Fortes
4. Achados Detalhados
   4.1 SEC-001 — Swagger UI exposto por padrão
   4.2 SEC-002 — Actuator sem restrição de detalhes
   4.3 SEC-003 — Grace period de refresh token
   4.4 SEC-004 — X-Forwarded-For sem proxy confiável
   4.5 SEC-005 — Validação de segredo incompleta
   4.6 SEC-006 — CSP com unsafe-inline em style-src
   4.7 SEC-007 — Rate limiting ausente em /notas
   4.8 SEC-008 — Versão Spring Boot inexistente
5. Recomendações Priorizadas
6. Plano de Reteste
7. Anexos
8. Issues para o GitHub

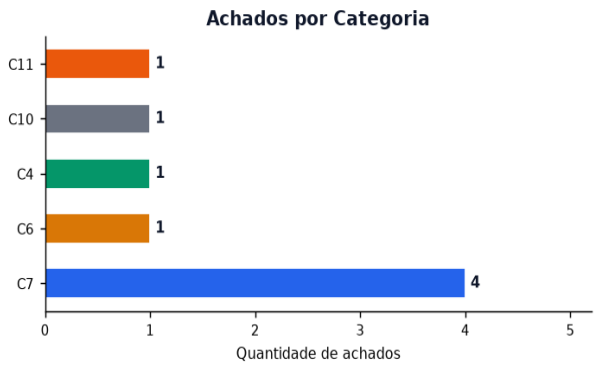
1. Resumo Executivo

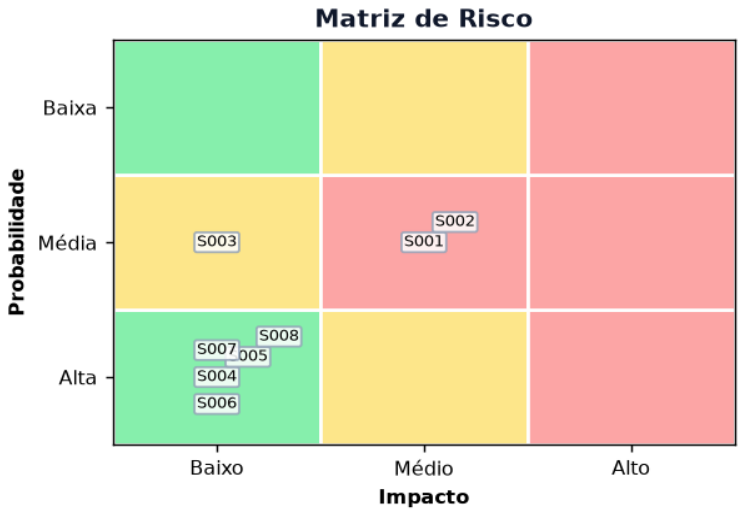
O projeto GENERICO demonstra uma **postura de segurança acima da média** para um template de aplicação web. Os mecanismos fundamentais — isolamento de tenant em dupla camada, armazenamento de segredos apenas como hash, criptografia em repouso do segredo TOTP, proteção de sessão com refresh token HttpOnly e revogação imediata — estão implementados corretamente e com atenção ao detalhe. Não foram identificadas vulnerabilidades críticas ou altas exploráveis diretamente a partir da análise do código-fonte.

Os **8 achados identificados** distribuem-se em faixas de risco médio a informativo, com ênfase em configurações de ambiente (Swagger/Actuator expostos, headers de proxy) que podem tornar-se problemas reais em um deploy de produção descuidado. O risco de negócio mais relevante é a exposição acidental do contrato completo da API e de informações de infraestrutura, que facilitam reconhecimento para ataques futuros.

Severidade	Qtd	Exemplos
Crítica	0	—
Alta	0	—
Média	0	—
Baixa	3	SEC-004, SEC-005, SEC-007
Informativa	2	SEC-006, SEC-008







3 Ações mais urgentes (P1/P2):

- Desabilitar Swagger UI em produção — definir SWAGGER_ENABLED=false como default e restringir no código (SEC-001).
- Adicionar management.endpoint.health.show-details=never no application.yml de produção (SEC-002).
- Expandir a validação de segredos de exemplo para MFA_ENCRYPTION_KEY na inicialização da aplicação (SEC-005).

2. Escopo, Metodologia e Limitações

Dentro do escopo

• Backend Java/Spring Boot: todos os pacotes em com.example.app (auth, security, tenant, audit, nota, shared, mail, config, user)
• Frontend Angular: src/app (core, features, layout, shared), configuração nginx
• Infraestrutura: docker-compose.yml, docker-compose.https.yml, Dockerfiles, scripts de banco (database/)
• CI/CD: .github/workflows/ci.yml, .trivyignore.yaml, dependências (pom.xml, package.json)

Fora do escopo

• Infraestrutura em execução, ambientes de produção/homologação
• Testes dinâmicos (DAST), engenharia social, segurança física
• Serviços de terceiros (Mailpit, GitHub Actions runners)
• Diretórios excluídos: node_modules/, target/, .git/

Stack detectada

Camada	Tecnologia
Backend	Java 21 · Spring Boot 3.5 · Spring Security · Spring Data JPA / Hibernate 6
Banco	SQL Server 2022 · multi-tenancy por coluna discriminadora (@TenantId)
Autenticação	JWT (JJWT 0.12.6) · BCrypt · TOTP (RFC 6238) · Refresh token opaco HttpOnly
Frontend	Angular 22 (standalone, signals, TypeScript strict) · Angular Material · Vitest
Infra	Docker Compose · nginx 1.31 · GitHub Actions · Trivy 0.58.0

Critério de severidade

Cada achado recebe um vetor CVSS v3.1 completo. O score base é ajustado pelo contexto: rotas acessíveis apenas a ADMIN, necessidade de autenticação prévia, ou mitigações compensatórias já presentes (ex: rate limiting, SameSite=Strict) são fatores de redução explicitamente declarados.

Limitações e premissas

• application.yml de produção não disponível: configurações como management.endpoint.health.show-details não puderam ser verificadas.
• Histórico git não foi varrido por segredos (requer git log --all -p — fora do escopo desta análise estática).
• Testes dinâmicos (DAST) não foram executados: algumas vulnerabilidades de configuração só são confirmáveis em runtime.

• Repositório Maven interno não verificado: versão 3.5.16 do Spring Boot não localizada no Maven Central.
• Configuração real de produção (WAF, firewall, HTTPS) não disponível para verificação.

3. Pontos Fortes

Os controles abaixo foram verificados no código e estão corretamente implementados. Eles representam a base sólida de segurança do projeto.

✓ **PS-001**	**Isolamento de tenant robusto em dupla camada**
Evidência:	tenant/TenantScopedEntity.java (@TenantId + @PostLoad), tenant/TenantScopedRepository.java (findById via JPQL), tenant/TenantHibernateConfig.java
Detalhe:	Camada 1: @TenantId do Hibernate filtra automaticamente todas as queries JPQL/derived. Camada 2: @PostLoad aborta caso EntityManager.find carregue linha de outro tenant. Sentinela NONE=-1 garante falha fechada quando TenantContext nao esta preenchido.

✓ **PS-002**	**Tokens opacos armazenados apenas como hash SHA-256**
Evidência:	shared/Hashing.java:sha256Hex(), auth/session/RefreshToken.java:token_hash, auth/token/TokenAcao.java:token_hash, auth/mfa/MfaRecoveryCode.java:codigo_hash
Detalhe:	Refresh tokens, tokens de acao (verificacao de email, reset de senha) e codigos de recuperacao MFA nunca sao gravados em texto puro. Comprometimento do banco nao revela os valores originais.

✓ **PS-003**	**BCrypt com salt para hashes de senha**
Evidência:	security/SecurityConfig.java:BCryptPasswordEncoder(), auth/AuthService.java:dummyHash
Detalhe:	Senhas em BCrypt. Anti-timing attack via dummyHash: mesmo quando o email nao existe, o tempo de resposta e equivalente ao de uma verificacao real.

✓ **PS-004**	**Segredo TOTP criptografado em repouso (AES-256-GCM)**
Evidência:	auth/mfa/SecretCipher.java
Detalhe:	IV aleatorio por operacao, tag de autenticacao de 128 bits, chave de 256 bits validada na inicializacao. Comprometimento do banco nao expoe segredos TOTP.

✓ **PS-005**	**Refresh token em cookie HttpOnly + SameSite=Strict + path restrito**
Evidência:	auth/RefreshCookie.java
Detalhe:	JavaScript nao pode ler o cookie. SameSite=Strict impede envio em requisicoes cross-site. Path=/api/v1/auth limita o escopo. Secure configuravel por ambiente.

✓ **PS-006**	**Access token exclusivamente em memoria no frontend**
Evidência:	frontend/src/app/core/auth/auth.service.ts:accessToken signal
Detalhe:	Token nunca escrito em localStorage/sessionStorage. XSS nao consegue extrair o access token de armazenamento persistente.

✓ PS-007 Validacao de segredo de exemplo na inicializacao (jwt.secret)

Evidência: config/AppProperties.java:Jwt record

Detalhe: Startup falha com mensagem clara se jwt.secret contiver o marcador 'troque'. Impede deploy acidental com segredo de exemplo.

✓ PS-008 Principio do minimo privilegio no banco de dados

Evidência: database/post/grants.sql

Detalhe: Usuario de runtime (app_runtime) tem apenas SELECT/INSERT/UPDATE/DELETE. DENY explicito em schema_version e UPDATE/DELETE em evento_auditoria. Sem DDL.

✓ PS-009 GlobalExceptionHandler nao vaza informacoes internas

Evidência: shared/GlobalExceptionHandler.java:unexpected()

Detalhe: Excecoes nao previstas retornam mensagem generica 'Erro interno'. Stack trace, SQL e mensagens de excecao ficam apenas no log com correlationId.

✓ PS-010 CorrelationIdFilter previne log injection

Evidência: shared/CorrelationIdFilter.java:isValid()

Detalhe: Header X-Correlation-Id aceito apenas se corresponder a regex [A-Za-z0-9\\-]{8,64}. Valores maliciosos sao substituidos por UUID gerado internamente.

✓ PS-011 Anti-replay TOTP via mfa_ultimo_passo

Evidência: auth/mfa/TotpService.java:verify(), user/Usuario.java:registrarPassoMfa()

Detalhe: Cada passo de tempo aceito e registrado. O mesmo codigo nao pode ser reutilizado, mesmo dentro da janela de tolerancia de +-1 passo.

✓ PS-012 Throttle distribuido persistido em banco

Evidência: auth/throttle/ThrottleService.java, auth/throttle/LoginTentativaRepository.java

Detalhe: Rate limiting funciona com multiplas instancias da API. Chaves por email (hash), por IP e por usuario. Bloqueio sobrevive restart da aplicacao.

✓ PS-013 Revogacao imediata de sessao verificada a cada requisicao

Evidência: security/JwtAuthenticationFilter.java:sessoes.estaAtiva()

Detalhe: Cada requisicao autenticada consulta o banco para verificar se a sessao continua ativa. Logout e revogacao valem instantaneamente sem esperar o JWT expirar.

✓ PS-014 Headers de seguranca HTTP no nginx

Evidência: frontend/nginx/security-headers.conf

Detalhe: HSTS, X-Content-Type-Options, X-Frame-Options: DENY, Referrer-Policy, Permissions-Policy e CSP configurados. server_tokens off.

✓ **PS-015**	**CI com varredura de vulnerabilidades (Trivy) em JAR e imagens Docker**
Evidência:	.github/workflows/ci.yml
Detalhe:	Trivy na versao fixa (0.58.0) varre JAR e imagens com --exit-code 1 para HIGH/CRITICAL. Execucao semanal agendada captura CVEs novos.

4. Achados Detalhados

ID	Sev.	CVSS	Arquivo principal	Título
SEC-001	Media	5.3	backend/src/main/java/com/example/app/security/SecurityConfig.java:52	Swagger UI exposto em producao por padrao
SEC-002	Media	5.3	backend/src/main/java/com/example/app/security/SecurityConfig.java:52	Actuator /health exposto sem autenticacao e sem restricao de detalhes
SEC-003	Media	5.9	backend/src/main/java/com/example/app/auth/session/SessionService.java:62-72	Grace period de refresh token permite reutilizacao em janela de tempo
SEC-004	Baixa	3.7	docker-compose.yml (FORWARD_HEADERS_STRATEGY: native)	FORWARD_HEADERS_STRATEGY=native sem lista de proxies confiaveis
SEC-005	Baixa	2.0	backend/src/main/java/com/example/app/config/AppProperties.java:20-24	Validacao de segredo de exemplo incompleta na inicializacao
SEC-006	Informativa	2.6	frontend/nginx/security-headers.conf:6	CSP do nginx usa 'unsafe-inline' em style-src
SEC-007	Baixa	4.3	frontend/nginx/locations.conf	Rate limiting no nginx ausente para endpoints de negocio
SEC-008	Informativa	3.7	backend/pom.xml:8	Versao do Spring Boot nao existe no Maven Central (3.5.16)

4.1 SEC-001 — Swagger UI exposto em producao por padrao

SEC-001	Swagger UI exposto em producao por padrao		MEDIA
Categoria	C7 - Configuracao Insegura	**CVSS Score**	5.3
CWE	CWE-16	**Vetor CVSS**	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP	A05:2021 - Security Misconfiguration	**Confiança**	Confirmado
Prioridade	P2	**Arquivos**	backend/src/main/java/com/example/app/security/Secu docker-compose.yml (SWAGGER_ENABLED default t

Descrição

A UI do Swagger (springdoc-openapi) e os endpoints /v3/api-docs/** estao liberados sem autenticacao em SecurityConfig. O docker-compose define SWAGGER_ENABLED=true como default, de modo que qualquer deploy que nao sobrescreva essa variavel expoe o catalogo completo da API publicamente.

Evidência (arquivo:linha)

```
.requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll() //
docker-compose.yml: SWAGGER_ENABLED: ${SWAGGER_ENABLED:-true}
```

Cenário de exploração

1. Atacante acessa https://alvo/swagger-ui.html sem credenciais. 2. Visualiza todos os endpoints, parametros, schemas de request/response e exemplos. 3. Usa essa informacao para planejar ataques

direcionados (enumeracao de endpoints, fuzzing de parametros, identificacao de versoes).

Pré-condições
Deploy sem sobrescrever SWAGGER_ENABLED=false. Backend acessivel pela internet.

Impacto
Vazamento do contrato completo da API facilita reconhecimento e acelera outros ataques.

Correção sugerida
1. Mudar o default em docker-compose.yml para SWAGGER_ENABLED=false. 2. Em SecurityConfig, condicionar o permitAll() a @ConditionalOnProperty(name='springdoc.api-docs.enabled', havingValue='true'). 3. Nunca habilitar em producao.

4.2 SEC-002 — Actuator /health exposto sem autenticacao e sem restricao de detalhes

SEC-002	Actuator /health exposto sem autenticacao e sem restricao de detalhes		MEDIA
Categoria	C7 - Configuracao Insegura	**CVSS Score**	5.3
CWE	CWE-209	**Vetor CVSS**	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
OWASP	A05:2021 - Security Misconfiguration	**Confiança**	Confirmado
Prioridade	P2	**Arquivos**	backend/src/main/java/com/example/app/security/Secu

Descrição
O path /actuator/health/** esta publicamente acessivel. A configuracao padrao do Spring Boot Actuator com management.endpoint.health.show-details nao restrita pode retornar informacoes sobre banco, disco e dependencias. Nao foi possivel verificar o application.yml de producao para confirmar se show-details=never esta definido.

Evidência (arquivo:linha)
```
.requestMatchers("/actuator/health/**").permitAll()
```

Cenário de exploração
1. GET /actuator/health/db (sem token). 2. Resposta pode revelar versao do banco, status de conexao e nome do datasource. 3. Informacao usada para direcionar ataques ao banco.

Pré-condições
show-details nao configurado como 'never' em producao.

Impacto
Divulgacao de informacoes internas de infraestrutura.

Correção sugerida
Adicionar ao application.yml: management.endpoint.health.show-details=never e management.endpoint.health.show-components=never. Restringir /actuator/health apenas ao path raiz (sem /**) ou usar management.server.port separado com firewall.

4.3 SEC-003 — Grace period de refresh token permite reutilizacao em janela de tempo

SEC-003	Grace period de refresh token permite reutilizacao em janela de tempo		MEDIA
Categoria	C6 - Autenticacao e Sessao	**CVSS Score**	5.9
CWE	CWE-613	**Vetor CVSS**	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:N
OWASP	A07:2021 - Identification and Authentication Failures	**Confiança**	Confirmado
Prioridade	P3	**Arquivos**	backend/src/main/java/com/example/app/auth/session/

Descrição

O refresh token e marcado como usado mas ainda e aceito para renovacao dentro da janela reuseGrace (configuravel via app.refresh.reuse-grace-seconds). Se um atacante obtiver o refresh token (ex: acesso fisico ao cookie, MitM em HTTP) dentro dessa janela, consegue criar uma sessao paralela sem disparar o mecanismo anti-replay. A injecao do token dentro do grace period nao aciona revogacao.

Evidência (arquivo:linha)

```
if (token.getUsadoEm() != null) { if (token.getUsadoEm().isBefore(now.minus(reuseGrace))) {
sessao.revogar(now); // so revoga APOS o grace period } } else { token.marcarUsado(now); }
```

Cenário de exploração

1. Atacante intercepta o cookie refresh_token de uma sessao ativa. 2. Vitima faz refresh normalmente; token e marcado como usado. 3. Dentro do grace period, atacante usa o mesmo token: a sessao NAO e revogada, dois access tokens ativos circulam. 4. Ambas as sessoes operam normalmente ate o proximo refresh apos o grace period.

Pré-condições

Atacante precisa obter o cookie HttpOnly (requer outro vetor: acesso fisico, comprometimento de proxy, subdomain cookie hijacking). Ataque viavel apenas durante o grace period.

Impacto

Sessao duplicada sem notificacao ao usuario.

Correção sugerida

Avaliar se o grace period e necessario para o produto. Se for, reduzir ao minimo (2-5s). Considerar alertar o usuario via auditoria quando o mesmo token for usado mais de uma vez, mesmo dentro do grace period. Documentar o risco aceito.

4.4 SEC-004 — FORWARD_HEADERS_STRATEGY=native sem lista de proxies confiaveis

SEC-004	FORWARD_HEADERS_STRATEGY=native sem lista de proxies confiaveis		BAIXA
Categoria	C7 - Configuracao Insegura	**CVSS Score**	3.7
CWE	CWE-348	**Vetor CVSS**	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N
OWASP	A05:2021 - Security Misconfiguration	**Confiança**	Requer validacao
Prioridade	P3	**Arquivos**	docker-compose.yml (FORWARD_HEADERS_STRAT

Descrição

Com FORWARD_HEADERS_STRATEGY=native, o Tomcat aceita X-Forwarded-For de qualquer origem sem validar se o remetente e um proxy confiavel. Se o backend estiver acessivel diretamente (sem passar pelo nginx), um cliente malicioso pode forjar o IP enviando X-Forwarded-For: 1.2.3.4, contornando bloqueios de throttle baseados em IP.

Evidência (arquivo:linha)

```
FORWARD_HEADERS_STRATEGY: native // RequestInfo.java: public String ip() { return
r.getRemoteAddr(); }
```

Cenário de exploração

1. Backend acessivel diretamente em :8080. 2. Atacante envia POST /api/v1/auth/login com header X-Forwarded-For: 192.168.1.1. 3. ThrottleService usa o IP forjado como chave: o bloqueio se aplica ao IP falso, nao ao IP real.

Pré-condições

Backend exposto diretamente sem o nginx como unico ponto de entrada. No docker-compose local, backend escuta em 127.0.0.1:8080 o que mitiga em desenvolvimento.

Impacto

Bypass parcial do rate limiting por IP em ambientes mal configurados.

Correção sugerida

Configurar server.tomcat.remoteip.internal-proxies com o range IP real do nginx/load balancer. Ou usar FORWARD_HEADERS_STRATEGY=framework com InternalProxyTrustedHeader. Em producao, garantir que o backend nao e acessivel diretamente.

4.5 SEC-005 — Validacao de segredo de exemplo incompleta na inicializacao

SEC-005	Validacao de segredo de exemplo incompleta na inicializacao		BAIXA
Categoria	C4 - Chaves Expostas	**CVSS Score**	2.0
CWE	CWE-798	**Vetor CVSS**	CVSS:3.1/AV:L/AC:H/PR:H/UI:N/S:U/C:L/I:N/A:N
OWASP	A02:2021 - Cryptographic Failures	**Confiança**	Confirmado
Prioridade	P2	**Arquivos**	backend/src/main/java/com/example/app/config/AppPr .env.example

Descrição

A protecao de startup que rejeita segredos com o marcador 'troque' existe apenas para app.jwt.secret (AppProperties.Jwt). As propriedades app.mfa.encryption-key e as senhas de banco nao tem validacao equivalente. Um deploy descuidado que copie .env.example sem substituir MFA_ENCRYPTION_KEY sobe a aplicacao com chave invalida (falha no Base64) ou, se o valor for trocado por algo fraco, sem rejeicao.

Evidência (arquivo:linha)

```
// AppProperties.java - valida APENAS jwt.secret: public Jwt { if (secret != null &&
secret.toLowerCase().contains("troque")) { throw new
IllegalArgumentException("app.jwt.secret nao pode conter marcadores..."); }} //
.env.example - MFA_ENCRYPTION_KEY sem validacao equivalente:
MFA_ENCRYPTION_KEY=troque-gere-com-openssl-rand-base64-32
```

Cenário de exploração

Desenvolvedor faz deploy com MFA_ENCRYPTION_KEY=chave-fraca-qualquer. A aplicacao sobe normalmente. Segredos TOTP ficam criptografados com chave fraca.

Pré-condições

Erro operacional no processo de deploy.

Impacto

Criptografia em repouso dos segredos TOTP enfraquecida sem alertas.

Correção sugerida

Adicionar validacao em AppProperties.Mfa: verificar se encryptionKey contem 'troque' (igual ao jwt.secret). Opcionalmente, verificar entropia minima (comprimento do Base64 decodificado = 32 bytes ja valida em SecretCipher, mas a excecao e RuntimeException generica sem contexto).

4.6 SEC-006 — CSP do nginx usa 'unsafe-inline' em style-src

SEC-006	CSP do nginx usa 'unsafe-inline' em style-src		INFORMATIVA
Categoria	C7 - Configuracao Insegura	**CVSS Score**	2.6
CWE	CWE-79	**Vetor CVSS**	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N
OWASP	A03:2021 - Injection	**Confiança**	Confirmado
Prioridade	P3	**Arquivos**	frontend/nginx/security-headers.conf:6

Descrição

A CSP permite estilos inline (unsafe-inline em style-src). Isso nao habilita XSS via script (script-src nao tem unsafe-inline), mas permite CSS injection, que pode ser usada para exfiltrar dados via seletores CSS ou alterar a aparencia da pagina de forma enganosa. O CLAUDE.md documenta que inlineCritical foi desabilitado justamente para evitar scripts inline na CSP.

Evidência (arquivo:linha)

```
add_header Content-Security-Policy "... style-src 'self' 'unsafe-inline' ..." always;
```

Cenário de exploração

Um atacante com capacidade de injetar HTML (ex: via stored XSS em campo nao tratado) pode usar <style> para exfiltrar tokens CSRF ou realizar UI redressing.

Pré-condições
Requer outra vulnerabilidade de injecao de conteudo.

Impacto
Baixo isolado; moderado se combinado com outra vulnerabilidade.

Correção sugerida
Avaliar remocao de 'unsafe-inline' de style-src e uso de nonces ou hashes para estilos criticos. Angular Material injeta estilos em runtime; pode requerer ajuste de build.

4.7 SEC-007 — Rate limiting no nginx ausente para endpoints de negocio

SEC-007	Rate limiting no nginx ausente para endpoints de negocio		BAIXA
Categoria	C10 - Abuso e Disponibilidade	**CVSS Score**	4.3
CWE	CWE-770	**Vetor CVSS**	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L
OWASP	API4:2023 - Unrestricted Resource Consumption	**Confiança**	Confirmado
Prioridade	P3	**Arquivos**	frontend/nginx/locations.conf

Descrição
O rate limiting por IP no nginx esta configurado somente para /api/v1/auth/. Endpoints de negocio como /api/v1/notas (POST/GET/DELETE) e /api/v1/admin/auditoria nao tem limite de requisicoes na camada nginx. O backend tem throttle apenas em operacoes de autenticacao; nao ha protecao equivalente para os endpoints de negocio.

Evidência (arquivo:linha)

```
# Apenas /api/v1/auth/ tem limit_req: location /api/v1/auth/ { limit_req zone=auth burst=20
nodelay; ... } # /api/v1/notas/ e /api/v1/admin/ nao tem limite nginx
```

Cenário de exploração
Usuario autenticado (ou token roubado) dispara milhares de requisicoes em /api/v1/notas para criar notas em massa ou sobrecarregar a lista. Sem limite nginx, a carga chega integra ao backend e ao banco.

Pré-condições
Token de acesso valido.

Impacto
Degradacao de disponibilidade; consumo excessivo de recursos de banco.

Correção sugerida
Criar zona limit_req_zone para endpoints autenticados (ex: rate=30r/s por IP) e aplicar em /api/v1/. Considerar limites por usuario no backend para endpoints de escrita.

4.8 SEC-008 — Versao do Spring Boot nao existe no Maven Central (3.5.16)

SEC-008	Versao do Spring Boot nao existe no Maven Central (3.5.16)		INFORMATIVA

Categoria	C11 - Dependencias	**CVSS Score**	3.7
CWE	CWE-1104	**Vetor CVSS**	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L
OWASP	A06:2021 - Vulnerable and Outdated Components	**Confiança**	Requer validacao
Prioridade	P2	**Arquivos**	backend/pom.xml:8

Descrição

A versao 3.5.16 do spring-boot-starter-parent nao existe no Maven Central (versoes publicas da linha 3.5.x vao ate 3.5.x patchlevel real). O build depende de cache Maven local ou repositorio interno para funcionar. Em CI sem cache, o build pode falhar silenciosamente ou usar uma versao incorreta. Requer validacao do repositorio Maven do ambiente.

Evidência (arquivo:linha)

```
<version>3.5.16</version> <!-- Spring Boot parent -->
```

Cenário de exploração

CI sem cache tenta baixar spring-boot-starter-parent:3.5.16, falha, e o artefato nao e construido.

Pré-condições

Sem cache Maven no runner de CI.

Impacto

Indisponibilidade de build; possivel uso de versao incorreta de dependencias.

Correção sugerida

Verificar e corrigir para a versao estavel real do Spring Boot (ex: 3.4.x ou 3.3.x conforme disponivel). Validar o pom.xml contra o Maven Central.

---

5. Recomendações Priorizadas

P1 — Imediato (< 1 semana)

Nenhum achado nesta prioridade.

P2 — Curto prazo (< 1 mês)

SEC-001 · Swagger UI exposto em producao por padrao · Esforço: Baixo/Médio

1. Mudar o default em docker-compose.yml para SWAGGER_ENABLED=false. 2. Em SecurityConfig, condicionar o permitAll() a @ConditionalOnProperty(name='springdoc.api-docs.enabled', havingValue='true'). 3. Nunca habilitar em producao.

SEC-002 · Actuator /health exposto sem autenticacao e sem restricao de detalhes · Esforço: Baixo/Médio

Adicionar ao application.yml: management.endpoint.health.show-details=never e management.endpoint.health.show-components=never. Restringir /actuator/health apenas ao path raiz (sem /**) ou usar management.server.port separado com firewall.

SEC-005 · Validacao de segredo de exemplo incompleta na inicializacao · Esforço: Baixo/Médio

Adicionar validacao em AppProperties.Mfa: verificar se encryptionKey contem 'troque' (igual ao jwt.secret). Opcionalmente, verificar entropia minima (comprimento do Base64 decodificado = 32 bytes ja valida em SecretCipher, mas a excecao e RuntimeException generica sem contexto).

SEC-008 · Versao do Spring Boot nao existe no Maven Central (3.5.16) · Esforço: Baixo/Médio

Verificar e corrigir para a versao estavel real do Spring Boot (ex: 3.4.x ou 3.3.x conforme disponivel). Validar o pom.xml contra o Maven Central.

P3 — Médio prazo (< 3 meses)

SEC-003 · Grace period de refresh token permite reutilizacao em janela de tempo · Esforço: Médio

Avaliar se o grace period e necessario para o produto. Se for, reduzir ao minimo (2-5s). Considerar alertar o usuario via auditoria quando o mesmo token for usado mais de uma vez, mesmo dentro do grace period. Documentar o risco aceito.

SEC-004 · FORWARD_HEADERS_STRATEGY=native sem lista de proxies confiaveis · Esforço: Médio

Configurar server.tomcat.remoteip.internal-proxies com o range IP real do nginx/load balancer. Ou usar FORWARD_HEADERS_STRATEGY=framework com InternalProxyTrustedHeader. Em producao, garantir que o backend nao e acessivel diretamente.

SEC-006 · CSP do nginx usa 'unsafe-inline' em style-src · Esforço: Médio

Avaliar remocao de 'unsafe-inline' de style-src e uso de nonces ou hashes para estilos criticos. Angular Material injeta estilos em runtime; pode requerer ajuste de build.

SEC-007 · Rate limiting no nginx ausente para endpoints de negocio · Esforço: Médio

Criar zona limit_req_zone para endpoints autenticados (ex: rate=30r/s por IP) e aplicar em /api/v1/. Considerar limites por usuario no backend para endpoints de escrita.

Quick wins destacados:

- SEC-001: Uma variável de ambiente (SWAGGER_ENABLED=false). Impacto imediato, esforço mínimo.
- SEC-002: Duas linhas no application.yml. Zero risco de regressão.
- SEC-005: Adicionar verificação de 'troque' em AppProperties.Mfa — < 5 linhas de código.

6. Plano de Reteste

Para cada achado, o critério abaixo define como validar objetivamente que a correção foi aplicada.

ID	Critério de aceite
SEC-001	GET /swagger-ui.html e GET /v3/api-docs sem token retornam HTTP 404 ou 403 em produção.
SEC-002	GET /actuator/health retorna apenas {"status":"UP"} sem subitens de componentes ou detalhes de banco.
SEC-003	Documentar o valor de app.refresh.reuse-grace-seconds e registrar como risco aceito na ADR. Opcional: monitorar eventos REFRESH_REUSO na auditoria.
SEC-004	Confirmar que em produção o backend não está acessível diretamente (apenas via nginx/load balancer). Verificar configuração de InternalProxies no Tomcat.
SEC-005	Iniciar a aplicação com MFA_ENCRYPTION_KEY contendo a palavra 'troque': deve falhar na startup com mensagem descritiva.
SEC-006	Verificar o header Content-Security-Policy na resposta do nginx: style-src não contém 'unsafe-inline'.
SEC-007	Confirmar que POST /api/v1/notas retorna HTTP 429 após exceder o limite de requisições configurado.
SEC-008	Build CI sem cache Maven completa com sucesso e usa a versão correta do Spring Boot.

Recomenda-se reexecutar esta auditoria completa após a aplicação das correções P1 e P2, e executar testes dinâmicos (DAST) em ambiente de homologação antes de qualquer deploy em produção.

7. Anexos

A — Tabela de Cobertura de Rotas/Handlers

Todas as rotas identificadas, com status de verificação de autenticação, autorização, verificação de posse/tenant, validação de entrada e resultado da auditoria.

Handler / Rota	Autent.	Autorização	Tenant/Posse	Validação	Status
POST /api/v1/auth/register	Não	Pública	N/A	Bean Validation	✓ Verificado
POST /api/v1/auth/login	Não	Pública	N/A	Bean Validation	✓ Verificado
POST /api/v1/auth/mfa/verify	Não	MFA token	N/A	Bean Validation	✓ Verificado
POST /api/v1/auth/refresh	Cookie	X-Requested-With	N/A	Cookie presente	✓ Verificado
POST /api/v1/auth/logout	Opc.	X-Requested-With	N/A	—	✓ Verificado
POST /api/v1/auth/logout-all	JWT	Autenticado	N/A	—	✓ Verificado
GET /api/v1/auth/me	JWT	Autenticado	N/A	—	✓ Verificado
POST /api/v1/auth/forgot-password	Não	Pública	N/A	Bean Validation	✓ Verificado
POST /api/v1/auth/reset-password	Não	Token de ação	N/A	Bean Validation	✓ Verificado
POST /api/v1/auth/verify-email	Não	Token de ação	N/A	Bean Validation	✓ Verificado
POST /api/v1/auth/resend-verification	Não	Pública	N/A	Bean Validation	✓ Verificado
GET /api/v1/auth/config	Não	Pública	N/A	—	✓ Verificado
POST /api/v1/auth/mfa/setup	JWT	Autenticado	userId token	—	✓ Verificado
POST /api/v1/auth/mfa/enable	JWT	Autenticado	userId token	Bean Validation	✓ Verificado
POST /api/v1/auth/mfa/disable	JWT	Autenticado	userId token	Bean Validation	✓ Verificado
POST /api/v1/auth/change-password	JWT	Autenticado	userId token	Bean Validation	✓ Verificado
GET /api/v1/notas	JWT	Autenticado	@TenantId	—	✓ Verificado
GET /api/v1/notas/{id}	JWT	Dono ou ADMIN	@TenantId	—	✓ Verificado
POST /api/v1/notas	JWT	Autenticado	@TenantId	Bean Validation	✓ Verificado
DELETE /api/v1/notas/{id}	JWT	Dono ou ADMIN	@TenantId	—	✓ Verificado
GET /api/v1/admin/auditoria	JWT	ROLE_ADMIN	tenantId token	Paginação limitada	✓ Verificado
GET /actuator/health/**	Não	Pública	N/A	—	■ SEC-002
GET /swagger-ui/**	Não	Pública	N/A	—	■ SEC-001

GET /v3/api-docs/**	Não	Pública	N/A	—	■ SEC-001

B — Categorias Não Aplicáveis

• C1 (Banco sem tranca): Aplicável — mecanismo verificado (@TenantId, TenantScopedRepository). Isolamento correto.

• C9 (SSRF/Upload/Desserialização): Não aplicável — não há endpoints de upload, requisições a URLs externas controladas pelo usuário, ou desserialização de dados não confiáveis.

• XSS no frontend: Não aplicável diretamente — Angular escapa por padrão. Não há uso de innerHTML/dangerouslySetInnerHTML. Único ponto: unsafe-inline em style-src (SEC-006, baixo risco).

C — Itens que Requerem Validação

• SEC-004: Verificar se o backend está exposto diretamente em produção (requer acesso ao ambiente).

• SEC-008: Verificar repositório Maven interno para confirmar existência da versão 3.5.16.

• Histórico git: varredura por segredos commitados (git log --all -p | truffleHog) não executada.

• application.yml de produção: management.endpoint.health.show-details não verificado.

D — Ferramentas Utilizadas

• Análise manual de código-fonte (white-box, estática)

• Kiro IDE — leitura e navegação de código

• Python 3.12 + reportlab + matplotlib — geração deste relatório

E — Glossário

Termo	Definição
CWE	Common Weakness Enumeration — catálogo de tipos de fraqueza de software.
CVSS	Common Vulnerability Scoring System — sistema de pontuação de vulnerabilidades (v3.1).
OWASP	Open Web Application Security Project — referência de riscos em aplicações web.
IDOR	Insecure Direct Object Reference — acesso direto a objeto por ID sem verificação de posse.
TOTP	Time-based One-Time Password (RFC 6238) — segundo fator de autenticação por aplicativo.
MFA	Multi-Factor Authentication — autenticação em múltiplos fatores.
RLS	Row-Level Security — isolamento de dados por linha no banco de dados.
DAST	Dynamic Application Security Testing — teste de segurança em aplicação em execução.
WAF	Web Application Firewall — firewall específico para aplicações web.
HSTS	HTTP Strict Transport Security — força uso de HTTPS pelo navegador.
CSP	Content Security Policy — política de segurança de conteúdo do navegador.

8. Issues para o GitHub

Texto completo de cada issue, pronto para copiar e colar no GitHub Issues. Achados informativos relacionados foram agrupados em uma única issue.

Issue #1 — SEC-001

Título: [Segurança] Swagger UI exposto publicamente por padrão (SEC-001)

Labels: security, severity:medium, config

```
## Descrição O endpoint `/swagger-ui.html` e `/v3/api-docs/**` estão configurados como públicos em `SecurityConfig.java` (linha 52) e o `docker-compose.yml` define `SWAGGER_ENABLED=true` como valor padrão. ## Por que é explorável Um atacante pode acessar o contrato completo da API sem autenticação, mapeando todos os endpoints, parâmetros e schemas. Isso acelera reconhecimento e fuzzing direcionado. ## Evidência ``` // SecurityConfig.java:52 .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll() // docker-compose.yml SWAGGER_ENABLED: ${SWAGGER_ENABLED:-true} ``` ## Referências - CWE-16: Configuration - OWASP A05:2021 – Security Misconfiguration - CVSS: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) ## Impacto Divulgação completa do contrato da API sem autenticação. ## Correção sugerida 1. Alterar default em `docker-compose.yml`: `SWAGGER_ENABLED: ${SWAGGER_ENABLED:-false}` 2. Em `SecurityConfig`, envolver o `permitAll()` em condicional `@ConditionalOnProperty` 3. Nunca habilitar em produção ## Critérios de aceite - [ ] `GET /swagger-ui.html` sem token retorna 403 ou 404 em produção - [ ] `GET /v3/api-docs` sem token retorna 403 ou 404 em produção - [ ] Variável `SWAGGER_ENABLED` não está no `.env` de produção ou está explicitamente como `false`
```

Issue #2 — SEC-002

Título: [Segurança] Actuator /health exposto sem restrição de detalhes (SEC-002)

Labels: security, severity:medium, config

```
## Descrição `/actuator/health/**` está público em `SecurityConfig.java:52` sem configuração de `show-details=never` verificada. ## Por que é explorável Com `show-details=always` ou `when-authorized` (default Spring Boot), a resposta pode expor versão do banco, status de disco e dependências internas. ## Evidência ``` // SecurityConfig.java:52 .requestMatchers("/actuator/health/**").permitAll() // application.yml: management.endpoint.health.show-details não encontrado no código ``` ## Referências - CWE-209: Information Exposure Through an Error Message - OWASP A05:2021 – Security Misconfiguration - CVSS: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N) ## Impacto Divulgação de informações de infraestrutura interna. ## Correção sugerida ```yaml # application.yml management: endpoint: health: show-details: never show-components: never ``` Considerar também restringir para apenas `/actuator/health` (sem `/**`). ## Critérios de aceite - [ ] `GET /actuator/health` retorna apenas `{"status":"UP"}` sem subitens - [ ] `GET /actuator/health/db` retorna 404 ou 403
```

Issue #3 — SEC-003

Título: [Segurança] Grace period de refresh token permite sessão duplicada (SEC-003)

Labels: security, severity:medium, session

```
## Descrição `SessionService.java:62-72` aceita um refresh token já marcado como `usado` dentro da janela `reuseGrace`. Se um atacante obtiver o cookie antes do dono usá-lo novamente, pode criar uma sessão paralela sem acionar revogação. ## Por que é explorável O mecanismo anti-replay só revoga a sessão quando o token é reutilizado **após** o grace period. Dentro da janela, duas sessões paralelas operam silenciosamente. ## Evidência ```java // SessionService.java:65-71 if (token.getUsadoEm() != null) { if (token.getUsadoEm().isBefore(now.minus(reuseGrace))) { sessao.revogar(now); // só revoga APÓS o grace period } // dentro do grace period: nenhuma ação, sessão duplicada ativa } else { token.marcarUsado(now); } ``` ## Referências - CWE-613: Insufficient Session Expiration - OWASP A07:2021 – Identification and Authentication Failures - CVSS: 5.9 (AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:N) ## Impacto Sessão duplicada sem notificação ao usuário. ## Correção sugerida - Avaliar se o grace period é necessário; se sim, reduzir ao mínimo (2-5s) - Registrar no AuditService sempre que um token for reutilizado dentro do grace period - Documentar como risco aceito em `docs/adr/` ## Critérios de aceite - [ ] Valor de `app.refresh.reuse-grace-seconds` documentado em ADR - [ ] Evento de auditoria gerado quando token é reutilizado dentro do grace period - [ ] OU grace period reduzido a ≤ 5 segundos
```

Issue #4 — SEC-004 / SEC-005 / SEC-007 / SEC-008

Título: [Segurança] Melhorias de configuração e hardening agrupadas (SEC-004, SEC-005, SEC-007, SEC-008)

Labels: security, severity:low, config, hardening

```
## Descrição Grupo de melhorias de baixo risco identificadas na auditoria estática: ### SEC-004 — X-Forwarded-For sem lista de proxies confiáveis `FORWARD_HEADERS_STRATEGY=native` em `docker-compose.yml` sem configurar `server.tomcat.remoteip.internal-proxies`. Se o backend for acessível diretamente, o IP pode ser forjado, contornando o throttle por IP. **Correção:** Configurar `server.tomcat.remoteip.internal-proxies` com o range IP do nginx/load balancer. ### SEC-005 — Validação de segredo de exemplo incompleta `AppProperties.java` valida `troque` apenas em `jwt.secret`. `MFA_ENCRYPTION_KEY` não tem validação equivalente. **Correção:** ```java // AppProperties.java — record Mfa public Mfa { if (encryptionKey != null && encryptionKey.toLowerCase().contains("troque")) { throw new IllegalArgumentException("app.mfa.encryption-key não pode conter marcadores de exemplo"); } } ``` ### SEC-007 — Rate limiting nginx ausente em endpoints de negócio `locations.conf` aplica `limit_req` apenas em `/api/v1/auth/`. Endpoints como `/api/v1/notas` não têm limite nginx. **Correção:** Adicionar zona de rate limit para `/api/v1/` com burst adequado. ### SEC-008 — Versão Spring Boot 3.5.16 inexistente no Maven Central `pom.xml:8` declara versão que não existe publicamente. Build pode falhar em CI sem cache. **Correção:** Atualizar para versão estável real do Spring Boot. ## Referências - CWE-348 (SEC-004), CWE-798 (SEC-005), CWE-770 (SEC-007), CWE-1104 (SEC-008) - OWASP A05:2021, A02:2021, API4:2023, A06:2021 ## Critérios de aceite - [ ] SEC-004: Backend não acessível diretamente em produção OU `internal-proxies` configurado - [ ] SEC-005: Startup falha com mensagem clara se `MFA_ENCRYPTION_KEY` contém 'troque' - [ ] SEC-007: POST `/api/v1/notas` retorna 429 após exceder limite configurado - [ ] SEC-008: Build CI sem cache completa com sucesso
```